Semantic Prioritization of Domain Data for Proactive Identification of Cybersecurity Indicators

Nic Recasens
Computer Science
Georgia Southern University
Statesboro, Georgia, USA
jr38088@georgiasouthern.edu

Nigel Smith
Computer Science
Georgia Southern University
Statesboro, Georgia, USA
ns15468@georgiasouthern.edu

***Abstract*—The identification of malicious domains requires separating cybersecurity indicators between valid and phishing URLs. This paper presents a machine learning pipeline designed to classify multiple malicious phishing and impersonation criteria, thus allowing a large proportion of malicious URLs to be proactively remediated. A robust lexical feature set was extracted from phishing URLs, including path depth and digit ratios. From there, to automatically establish a basis for impersonation, we utilized cosine similarity scores between Phishtank's known phishing URLs and the Tranco legitimate domain dataset, applying a 75th-percentile threshold on semantic similarity to create a binary target variable [5]. This target represents mined and cleaned semantic metadata. From there, two AI methodologies were applied to this: the first was a K-Means clustering model for overall extraction of malicious methodologies. After that, we honed in on the attack methodology of "impersonation" with a Support Vector Machine. It's RBF kernel was trained and optimized via randomized grid search to reach the "proactive" goal of our paper, with it being able to work on novel data. This allowed ML classification to pick up after the semantic extraction and automatically classify domains of note. Performance for all models was rigorously evaluated to ensure efficacy, and their results were correlated in a human-readable triage tool. Those deliverables showed that our methodology successfully demonstrated that structural URL characteristics can effectively capture impersonation intent. With tools like this, semantic classification tasks like impersonation detection are very achievable, with lexical ML being able to reliably act upon the remaining data effectively.**

I. Introduction and Problem Statement

In the modern world, attackers are a step ahead of all Cybersecurity professionals. Professionals need to be able to adapt to every threat out there, while attackers only need to be able to adapt to a single fatal weakness. As such, the realm of machine learning and predictive analytics is of known importance in Cybersecurity research. For decades, machine learning analysis has helped keep defenders on top of their game by preventing novel attacks from gaining footholds. In the 90s, when the number of attackers was smaller, professionals were able to get away with only blocking known threats. But with Ransomware and Malware becoming billion dollar multi-national black markets, we need to be able to apply risk trends to novel data. At the forefront of this prevention is the identification and blocking of spam and phishing. An individual datum related to spam and phishing is called an indicator of attack (IoA), and they let us know if someone is targeted by an attacker. And if they are used successfully, they become indicators of compromise (IoC). IoCs have the negative reputation of being indicative of failed Cybersecurity protection, but in relevance to this project, they are the second half of the puzzle that is semantic prioritization of indicators. Via the correlation of IoAs and/or IoCs, numerous researchers have pushed the boundary forward over the decades since the aforementioned 1990s. The attackers may be numerous in their successes, but they leave behind their own value as well: data. With that data, we can see how pervasive cyber threats are, train models to simulate said attackers, and allow the very computers they attack help mitigate the risks themselves. Our team looked at these untapped indicators, and developed our research questions: Is there more data to be mined from the individual datum that is a phishing domain? What more can be done with these foundational data points now that Natural Language Processing (NLP) has grown so rapidly in so few years? Is there a correlation between domain's lexical and semantic metadata? And critically, our guiding research question was this: **Do domain semantics help predict real-world phishing and impersonation activity to a statistically significant degree?** Machine learning and data science stand on the forefront of successful action upon IoCs, and throughout this paper, we will show how important cyber-risk mitigation truly is in our modern interconnected world

II. Related Literature

A. Literature Selection Methodology

At the most basic level, the 3 keywords we used to guide our literature selection process were "spam", "phishing", and "semantic". Spam and Phishing are the strongest indicators Cybersecurity professionals have as indicators of attack. In fact, Business Email Compromise or (BEC) can be identified as the largest threat to the modern digital business [13]. As such, once we have papers scoped to the huge risk spam and phishing compose, the "semantic" keyword allows us to predominantly find Computer Science papers detailing how semantic analysis can help with this cause. Some linguistic journals do appear with these keywords, but with the current rise of the Large-Language Model, semantic research is seeing a heavy Computer Science edge.

For searching these terms, Google Scholar was the primary resource, though GALILEO also was utilized for accessing some non-free files. When on our Georgia Southern University network, some research journals allow our IP to access full papers without logon, which was also useful for reading full texts. Publication Year and citation information was considered and prioritized past the above keyword filtering, and was based on the Google Scholar data for those indicators.

And to wrap up our selection process, our primary exclusion criteria was mentioned earlier, and that is a Computer Science focus. Some articles were industry paid, so were more like Sociology in nature. And for other articles, they were focusing on the behavioral aspects of phishing and spam, which while useful, are not in scope for this review. And speaking of scope, the primary *inclusion* criteria was related to this course's guidelines, and that is publication year. Many useful articles at the beginning of semantic analysis for spam prevention were made in the late 2000's, but they were specifically asked to be ignored for this paper by the paper requirements. As such, only 'recent' research was provided, meaning publication dates past 2020 as much as it could be helped. These criteria, alongside with the above requirements and toolset mindsets, allowed us to choose the below 8 papers to systematically review.

B. Systematic Literature Review

1) Reactive Detection of Attacks: The earliest theme in the research of IoCs addresses detection of spam and phishing *after* it occurs. Saidani, Adi, and Allili [13] confront the fact that spam continues to account for over 56% of total email traffic, yet traditional detection systems largely rely on the provenly ineffectual "bag-of-words" model, which evaluates emails as an unstructured collection of independent words without considering grammar, context, or word order. Spammers purposefully exploit the weaknesses of basic keyword filters through obfuscation, motivating their proposed two-level semantic classification framework, which achieved up to 98% accuracy across domain-specific classifiers [13]. Sonowal and Gunikhan [14] extend this challenge to SMS, noting that unlike email's 20-30% open rates, SMS messages boast a staggering 98% open rate, making smishing a disproportionately dangerous vector. Their feature-based approach reduced 52 input dimensions to just 20 via Kendall rank correlation (a 61.53% decrease) while maintaining 98% accuracy, a reduction critical for viability on resource-constrained mobile devices [14]. Both papers share a common vulnerability: as generative AI tools become more indistinguishable when mimicking professional readability, the semantic dissonance these systems rely on may diminish [14].

2) Proactive Data Gathering: A second thread in the literature pushes beyond reactive detection toward proactively building threat intelligence. Zhang et al. [16] identify that before spam and phishing can be remediated, we have to know what they actually *are* via data collection, noting that conventional CTI relies on "fixed-point" monitoring of curated feeds that is time-consuming, delayed, and incomplete. Their iMCircle system addresses this with an active recursive architecture that mines IoCs continuously from open-source web data using NLP and SVM classification, achieving 96.70% precision and an F1 score of 90.46% [16]. However, the system remains entirely dependent on publicly indexed information and cannot surface threats not yet discussed on the open web [16]. Zieni, Massari, and Calzarossa [17] complement this by surveying the broader detection landscape, finding not a single technical gap but a fragmentation of approaches across list-based, similarity-based, and ML-based methods. Their survey identifies Random Forest as the most consistently performant traditional algorithm, while also flagging a critical research gap: URL shortening services render most lexical URL features meaningless [17].

3) Proactive Prevention of Attacks: The most recent papers in the review target prevention rather than detection, driven by capabilities unlocked by large language models and self-updating ML systems. Brezeanu, Archip, and Artene [1] observe that phishing kits (reusable HTML templates that allow attackers to rapidly deploy fraudulent pages) frequently reuse identical HTML structures across campaigns, making the underlying code structure a more reliable detection signal than visual or textual content. Their Phish Fighter system applies agglomerative hierarchical clustering and a self-retraining Random Forest classifier, achieving weighted precision, recall, and F1 scores exceeding 90% [1]. Desolda, Greco, and Viganò [7] address a parallel problem: that no existing system achieves perfect accuracy, meaning borderline cases reach the user, and the warnings users actually see in browsers and email clients are widely ineffective. Their APOLLO system, powered by GPT-4o, achieved 97.4% accuracy without hand-crafted features, rising to 99.9% when mature VirusTotal data was available [7]. Critically, both systems still encounter the zero-day ceiling: Phish Fighter requires a minimum of three pages from a new kit before clustering, and APOLLO's performance worsens when VirusTotal returns uncertain data on newly minted URLs [1], [7]. These are key areas where domain semantics may be able to shine in our research.

4) Psychological Studies: No technical system operates in a vacuum, and as such, there is a robust category of research evaluating the factors affecting phishing attacks once they become a false negative. Dash and Ansari [4] note that despite advancements in defensive systems, "naïve information security practices" account for between 72% and 95% of all cyber threats, motivating their study of AI-enabled training tools like viCyber, which can provide real-time feedback and adapt to each user's understanding level [4]. Sarno, Harris, and Black [6] ground this in empirical data, finding that some phishing emails will inevitably reach the inbox, at which point the user becomes the last line of defense. Their regression study of 150 participants found that younger adults were most susceptible (directly contradicting the popular assumption that older adults are the most vulnerable population) and that self-reported phishing experience may foster overconfidence rather than genuine detection skill [6]. Together, these papers reinforce that interpretable outputs and end-user empowerment are not secondary concerns but core requirements for any

detection system, including the detections proposed by our team's research.

C. Overall Literature Synthesis

Two common themes reveal themselves across these papers. First, existing tools are commonly vulnerable to zero-day attacks, which suggest the need for robust ML and deep learning tools. From the reactive blocklists reviewed by Zieni et al. [17] to the phishing kit clustering of Brezeanu et al. [1] to the retrieval-augmented classification of APOLLO [7], every system reviewed encounters the same fundamental constraint: detection accuracy degrades sharply when the threat has not been seen before. Brezeanu's Phish Fighter requires a minimum of three pages from a new kit before it can form a cluster, and APOLLO's performance actually worsens when VirusTotal returns uncertain data on newly minted URLs. Even iMCircle, which was designed specifically for continuous discovery, can only surface indicators that have already been publicly discussed somewhere on the open web [16]. The zero-day problem is not merely a limitation noted in passing by these authors; it is the central unsolved challenge that connects the entire body of work reviewed here. Second, the ability to meet this need exists, but is still in its infancy, with effective ML tools having only existed for a matter of a few years. The progression visible across the papers is striking: Saidani et al. [13] advanced past bag-of-words with domain-specific semantic rules in 2020, Sonowal and Gunikhan [14] showed that careful feature engineering could make ML viable even on resource-constrained mobile devices, and by 2025, APOLLO demonstrated that a general-purpose LLM could classify phishing emails at 97.4% accuracy without any hand-crafted features at all [7]. Our project aims to reconcile the findings and shortcomings of these papers by utilizing cutting-edge ML tools for the purpose of highly accurate and immediate phishing detection. Specifically, the research gaps identified here point toward needing a system that can operate effectively against novel threats without requiring a warm-up period of previously seen IoCs, that can leverage the semantic reasoning capabilities demonstrated by LLM-based approaches while maintaining the structural analysis strengths shown by methods like Phish Fighter, and that produces outputs interpretable enough to support the human end user who remains, as this review has shown, the last and most variable line of defense. We aim to contribute to these gaps by looking at the most fundamental datum of phishing datasets (domains) in order to expand upon the previously unanalyzed base semantic data that underpins modern cybersecurity blocking. This is through extracting these domains semantic information, and using it to prioritize phish defense in a novel way.

III. Dataset and Preprocessing

To identify indicators, we need both the indicators themselves, but also a control group, a set of data that we would consider an "Indicator of Integrity", or IoIs. IoIs let us see how we expect a legitimate domain to behave and be shaped like in our data. So with that, two data sources were chosen

```
Avg PhishTank URL length : 71.2
Avg PhishTank host length : 20.2
Avg Tranco domain length : 12.3
Median PhishTank URL length : 38
Median Tranco domain length : 12
Mean hyphens - PhishTank : 0.93
Mean hyphens - Tranco : 0.09
Mean digit ratio - PhishTank : 0.1023
Mean digit ratio - Tranco : 0.0124
PhishTank unique URLs : 55,625
PhishTank unique domains : 8,271
Tranco unique domains : 50,000
```

Fig. 1. Data Shape from Exploratory Data Analysis

by our team. For IoCs, the Phishtank Verified dataset was chosen, as it had a reasonable amount of training data with 50k+ Phishing URLs verified as malicious by Cisco Talos [2]. And for trustworthy domains, Tranco Top 1 Million was used. Tranco tracks and aggregates the 1 million most used sites, and provides them as a research dataset [11]. These 2 combined allow us to glean basic lexical and numerical insights on the data typically used by spam detection. Figure 1 shows how stratified the 2 datasets can be with their features.

As expected, the data had to be cleaned and normalized for us to do meaningful analysis. All of our data from that cleaning is stored on Google Cloud Storage Containers and GitHub, primarily to achieve the goal that our results can be replicated by our professor. Cloud storage had to be used because some of our data is so large that embedding it in the paper or code would be unreasonable, and additionally, neither the Phishtank nor Tranco datasets can be downloaded unauthenticated from their original sources. We used Colab and Python to do analysis, with Pandas, Polar, MatPlotLib, and Seaborn as our foundational tools. All of these are industry standard, and allow our paper to have repeatable, easily interpretable code.

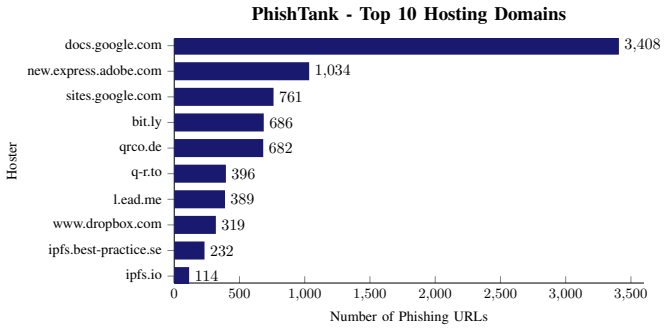

Fig. 2. Showcase of Deduplication Needs

In terms of cleaning, the primary work is deduplication. A domain like "google.com" may have URLs of "https://www.google.com", "http://www.google.com", or even just minor differences like "google.com/" with a trailing slash at the end [2]. And for the IoCs specifically, they have an issue of a large chunk of them being hosted by a legitimate (or at

least largely used) hosting platform. As shown in Figure 2, many attackers use the perceived legitimacy of larger sites to hide in the crowd, and to create malicious payloads that HTTP based scanners may not be able to identify quickly. This means when we train our ML later, we will need to filter out these hosting domains.

And past deduplication, both datasets have lots of auxiliary info that while useful, would not reasonably be able to be embedded. This occurs when we can't be assured that real world attacks would have said auxiliary data. So while Phishtank gives metrics like intended target, that can be useful for Exploratory Data Analysis (EDA), but they are not in scope for our work. This means that from the raw data, the primary cleaning stages would be stripping irrelevant info for our purposes (like leading slashes), and focusing only on the domains themselves.

Once a clean set of domains was grabbed, our team enacted "Text Embeddings" on the dataset using the `gemini-embedding-001` model available on the Google Cloud Platform. Embedding is the process of turning regular text into multi-dimensional vectors, in which processes like Cosine similarity and Euclidean distance can see how "close" the vectors are to each other [13]. Our code base (expanded upon in the appendix) shows how we utilized Google's cutting edge Embedding platform. Since these vectors represent encoded semantic data, "closeness" of vectors correlates to closeness of semantic meaning, letting us extract answers from the data using semantic reasoning. To note, training our own embeddings / semantic model from scratch was not the goal nor scope of our project, but rather using data science to analyze the semantic depth behind the domains once embedded. That is why we were happy to use the pre-trained 3,072 dimensional Google Gemini embeddings framework [9].

Once embeddings are made and compared, they will have to be stored and aggregated, as the raw results of our team's cosine similarity data came out to 23GB of data, with over 2.7 billion vector comparisons. That is too much to do in RAM alone on Colab, so we process the data line by line and store to disk. From there, we did exploratory correlation, classification, and novel data testing. In the next section we will glean insights from this on how the data's semantic layer adds on to the capabilities exposed in the literature review. Table 1 shows the basic shape of the embedded AI data as a preview of this analysis. With modern tools like vector databases, we could see already effective tools like iMCircle and Apollo gain a new semantic level of data for automatic decision making with these embeddings [7], [16]. But for our process, we will use the closeness / similarity of the vectors as a filter, a sort of "sift" that allows us to train an SVM on the URL and it's features when a semantic hit occurs on the URL. Figure 3 shows all of this in a formalized Framework Diagram. The explained steps above each feed into each other, and through our diligent following of the outlined framework, our team has the resources to provide answers and context to our research question about the efficacy of domain semantics.

Top 50k Domain	Top PhishTank Match	Cosine Similiarity
chateauversailles.fr	chateauversalles.fr	0.92593
tdsynnex.com	tdsynnexdeveloper.com	0.924352
daiwa.co.jp	daiwa.jp	0.922306
tronlink.org	tronlink.org.uk	0.922023
bradescoprime.com.br	bradesco.primeoficial.com	0.920717
sncf-connect.com	sncf-connect.app	0.918135
scrtc.com	scrtc.weebly.com	0.915968
lassuranceretraite.fr	lassuranceretraite.net	0.914879
t-2.net	t-2net.com?	0.91394
hushmail.com	hushmail.web.app	0.912192

TABLE I
TOP 10 SEMANTICALLY SIMILAR DOMAINS BETWEEN DATASETS (EXAMPLE OF EMBEDDINGS DATA USE CASE)

IV. METHODOLOGY

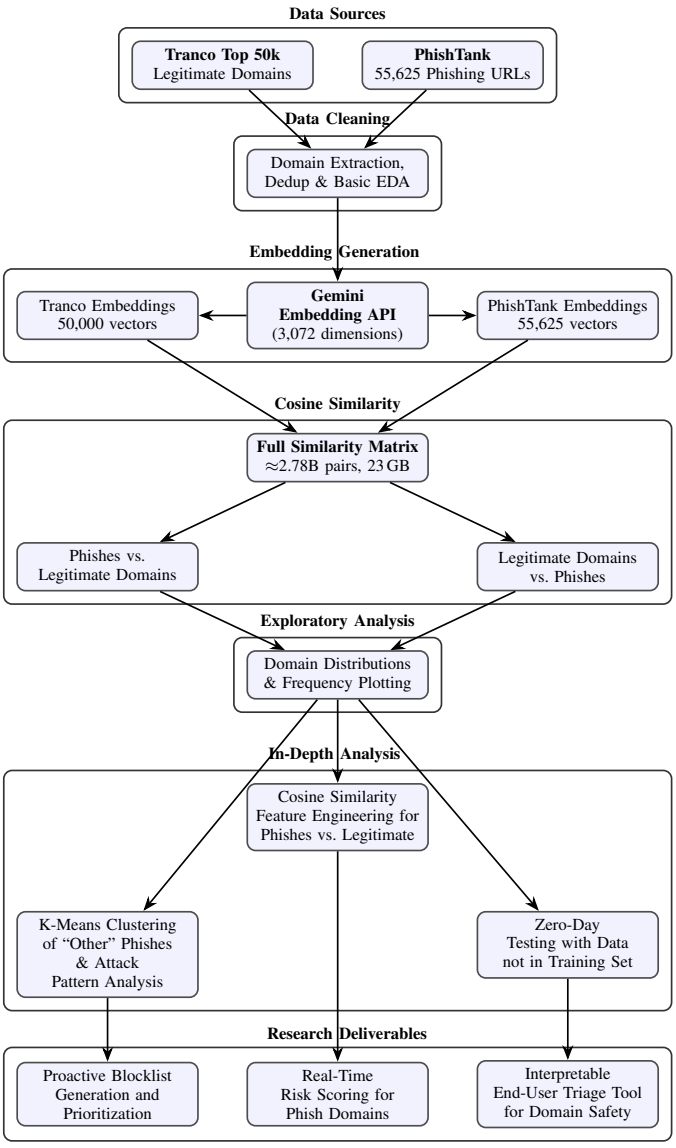

Fig. 3. Domain Semantics Extraction Framework

A. Feature Engineering

Raw URLs pulled from the Tranco and Phishtank datasets do not come labels such "impersonation likelihood" by default; at best (in the case of Phishtank) there is the Target attribute, but a mere 10% of tuples have a Target attribute that is not simply labeled as "other". As such, at the most foundational end, we started with clustering. This unsupervised approach would see if lexically, any additional meaning can be extracted from the domains that PhishTank did not already have in its feature set. From there, we expanded the scope to look from lexical clusters to semantic ones, with the specific cluster of "impersonation" being our goal label. If Clustering and Semantic Impersonation detection both work, that shows that there is both semantic and lexical depth to domains that is being underutilized. Additionally, by showing how impersonation can be trained out of the data, we can see how each lexical cluster could lead into a corresponding semantic model for that risk factor.

To achieve all of this, we utilized the cosine similarity data to implement the impersonation label. If a URL scored above the 75th percentile for cosine similarity, it was marked as an impersonating domain. 75th percentile was chosen to balance precision and coverage so as to minimize generic URLs getting flagged (false positives) and true impersonations slipping through the cracks (false negatives). There is valid consideration to be given to tweaking this parameter such that false negatives are weighted higher; real world usage may likely prefer the model to be overly sensitive and flag safe site erroneously than let a malicious site through [14].

B. Model Selection

For our SVM, we then decided to go with a classic Support Vector Machine as the model to work with. SVMs are distance-based learners, so this means our model is particularly sensitive to feature scale. This meant we also encapsulated the data with StandardScaler and SVC within a scikit-learn pipeline, with both helping to ensure consistency [8]. The overall decision to go with an SVM comes from the desire to seek correlation between the semantic weight of a domain and its maliciousness. As such, model complexity or pure optimization was not the goal, but rather a proof of concept, which an SVM more than achieves. We selected the Radial Basis Function (RBF) kernel as our primary estimator. This is because while we initially benchmarked a linear model, the RBF kernel produced superior results. Since RBF is a good baseline for ML anyways, the choice to stick with RBF was clear. And for clustering, we decided to go for K-Means. This follows much in the same inspiration as we had for using an SVM, a straightforward model methodology to see if there is merit to the research question: are there categories to be extracted from the lowest level datum?

C. Experimental Setup

To prepare the data, past the above scaling and aforementioned cleaning, we We implemented a stratified 80/20 train/test split to ensure that our class distribution remains consistent across both subsets. We need to stratify the data in our case since the PhishTank dataset isn't evenly distributed when it comes to impersonation attempts. To account for our 75/25 class imbalance, we utilized the `class_weight='balanced'` parameter. This automatically adjusts the cost function to place a higher penalty on misclassifications of the minority class, preventing the model from developing any sort of inappropriate bias towards the majority class. And lastly, rather than using an exhaustive and time consuming grid search, we employed Randomized Search with 5-fold stratified cross-validation. This cross-validation and tuning process works by sampling a fixed number of parameter combinations from the search space. By implementing these, we achieved a significant tuning speed-up while still receiving the benefits of tuning our model in the first place. We initially did some attempts to use grid search, but pivoted after iteration time took too long for user-led tuning.

As mentioned in the previous section, Scikit-Learn was chosen as our coding library, with Python 3.14 over Google Colab being our processing. All of these are industry standard choices for base level research, and worked together well for our purposes [3], [10]. For our imports, the below annotated code shows our usage, and our reasoning for each. These imports are the glue and facilitators of our above model considerations in the code.

```
# URL data ingest for handling OS paths &
    regex & phish domains
from pathlib import Path; from urllib.parse
    import urlparse
# For Exploratory Data Analysis (EDA), vector
    math, and making sense of distributions
import pandas as pd; import numpy as np;
    import matplotlib.pyplot as plt; import
    seaborn as sns
# Model Selection & Cross-Validation for
    splitting data into sets and searching for
     the best hyperparameters
from sklearn.svm import SVC; from sklearn.
    model_selection import train_test_split,
    RandomizedSearchCV, StratifiedKFold
# Preprocessing & Feature Scaling for ensuring
     features are on a similar scaling for ML
from sklearn.preprocessing import
    StandardScaler
# Meeting the best practice of pipelining our
    ML model
from sklearn.pipeline import Pipeline
# And finally, these imports are for
    quantifying how well the model performed
from sklearn.metrics import (
    classification_report, confusion_matrix,
    ConfusionMatrixDisplay, roc_curve, auc,
    RocCurveDisplay)
```

Fig. 4. Import Code from Appendix C: SVM ML Showing Our Implementation Logic

V. Experiments and Results

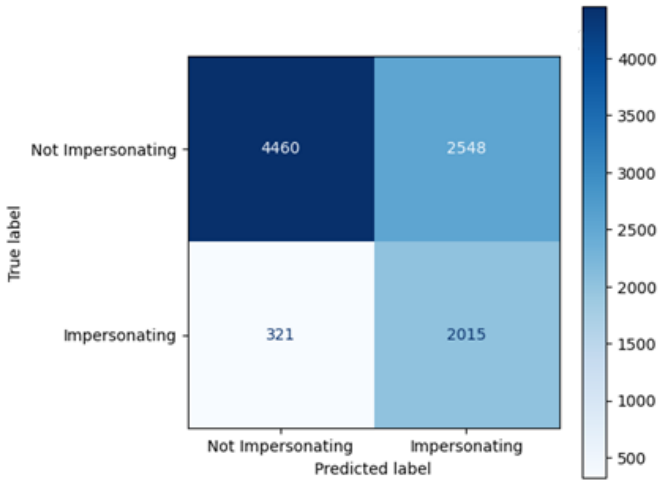

Fig. 5. Confusion Matrix for Tuned RBF SVM

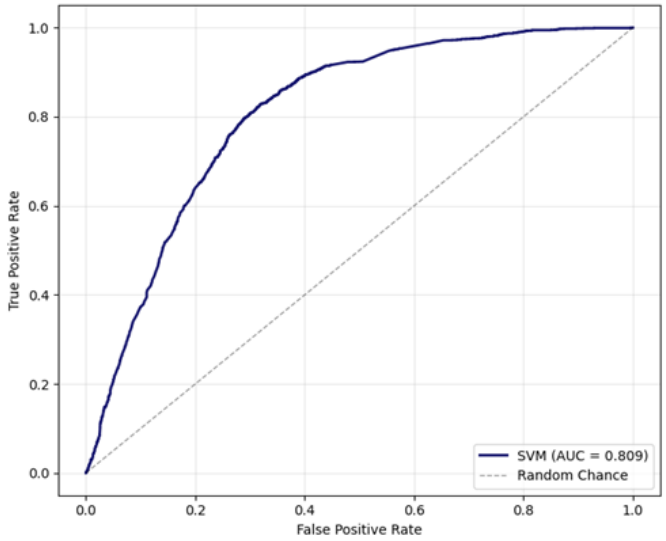

Fig. 6. ROC Curve for Tuned SVM

A. Primary Results

The confusion matrix visual shows the prediction accuracy of the tuned SVM model by mapping its classifications against the actual data established in previous steps. Data reveals that while the described model successfully identifies a significant number of threats (2,015 True Positives), it also produces a high volume of False Positives (2,548), frequently flagging standard phishes as impersonation. This is important because it quantifies our model's purposefully "aggressive" tuning, showing a clear preference for catching more potential phishes at the cost of a higher false alarm rate. Our Classification model in this case could do great for our goal of prioritization, making it so rather than processing all of the data for more in-depth tests, using a model like ours to trim the fat. That means we have reached the goal of our main research question, our model can distinguish well enough to prioritize and highlight domains that need more analysis, showing value in the semantic analysis.

B. Qualitative Analysis

Our ROC Curve visual for the Tuned SVM shows an AUC of 0.809, which is more than decent for our timescale and first approach. The model proves to be a solid classifier, correctly ranking an impersonation domain higher than random chance in over 80% of cases. The significant gap between the blue curve and the "Random Chance" line confirms that the features we previously examined (like host length and path depth) are doing more than just guessing. This also achieves our research question goal of statistical significance, as beating random chance establishes that.

C. Clustering Results

K-Means clustering analysis was performed on all data points labeled as "Other" in the PhishTank dataset; these account for approximately 90% of all ~56,000 data points. Four groups were chosen based on the highest number of clusters that still introduced minimal noise. As seen in the elbow curve, a case can be made for `k=3`, as two clusters show conceptual similarities; however, their feature profiles are distinct enough to warrant separation.

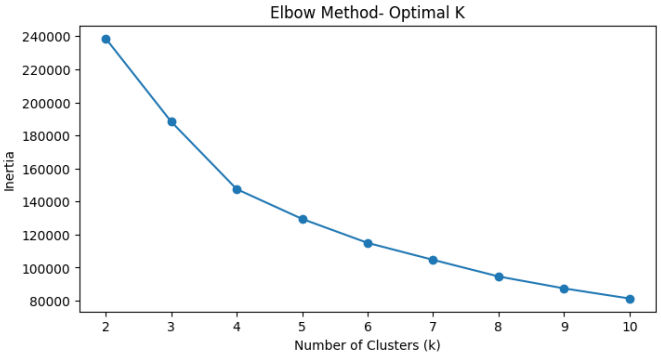

Fig. 7. Elbow Curve for Baseline Lexical Clustering

The four groups identified are as follows:

- **Cluster 0 (7,235 URLs) — Platform Abuse (Simple):** Phishing pages hosted on cloud storage and form services such as Jotform and Cloudflare R2. Characterized by long hostnames and high digit ratios, commonly from auto-generated subdomain strings.
- **Cluster 1 (21,341 URLs) — Platform Abuse (Complex):** Attacks hosted on free website builders such as

Google Sites, Weebly, and Wix. Characterized by moderate host lengths and low digit ratios, making them harder to differentiate from legitimate traffic due to their domain similarities to legitimate sites.

- **Cluster 2 (8,680 URLs) — Token/Long URL Abuse:** Instances of legitimate platforms such as Google Docs or Dropbox hosting malicious content, identifiable by high path depth and long authentication token strings.
- **Cluster 3 (15,517 URLs) — Shortener/Redirector:** URLs concealed behind redirect services, characterized by very short hostnames such as `qrco.de` or `t.co`.

Clusters 0, 1, and 2 together account for over 70% of the "Other" data points. This directly validates the concern raised by Zieni et al. [17] regarding the use of legitimate hosting infrastructure to house phishing content, and confirms that standard lexical detection alone is unreliable against this dominant class of attack. The successful results of the SVM above show that training ML on the semantics of each of these clusters is the best way to get both meaningful extraction and semantic prioritization.

VI. Discussion

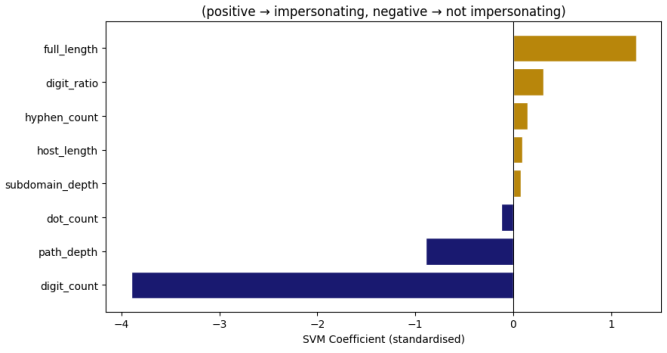

Fig. 8. Linear SVM Feature Coefficients

A. Interpretation of Results

Figure 8 shows us a critical summary of our ML's results. This figure is us using another trained SVM to mine the coefficients from our RBF weights (as is industry standard [12]). We have to do this extra SVM step because RBF weights are not human readable on their own. The visual gives a clear ranking of which URL features most influenced the final classification. We can see that `full_length` and `digit_ratio` are the primary drivers toward an "impersonation" label, while a high `digit_count` strongly pulls the prediction toward "not impersonating" in this specific dataset. This lets us infer too high of a digit count could indicate a more generic phishing attack. The ability to correlate lexical features with semantic prioritization is a great success. This correlation is essential because it moves the model from a "black box" into an interpretable tool, showing exactly which features the algorithm prioritizes. This allows data science work like ours to audit the model's logic and ensure it is focusing on the most relevant structural anomalies found in phishing URLs.

B. Comparison with Prior Work

Our SVM achieved an AUC of 0.809 operating purely on URL lexical features, which is competitive with the broader detection landscape surveyed by Zieni et al. [17], who identified Random Forest as the most consistently performant traditional approach. Notably, our pipeline requires neither page content nor third-party intelligence feeds: unlike Phish Fighter [1], which requires HTML structure, or APOLLO [7], which degrades without VirusTotal data, our approach functions on the URL string alone. The semantic similarity layer that generates our impersonation label is also novel relative to prior work; rather than relying on human-annotated labels, we mine a proxy target from embedding-space proximity, a method not employed by any of the surveyed systems. Our clustering results further add a categorization dimension that reactive systems do not provide, offering analysts a characterization of attack type rather than a binary verdict. These two in tandem offer a full view of the data able to be mined from a phishing URL.

C. Limitations

Three primary weaknesses do arise here however. First, the impersonation label is derived from an embedding-based cosine similarity threshold, not from human annotation. The quality of this proxy label depends on the embedding model's fidelity, and the assumed semantic intent. Next, PhishTank and Trancos both are not perfect data sources. PhishTank is community driven and not well cleaned, meaning our cleaning has the potential to not fully mitigate error. And Trancos does not provide only safe domains, but rather popular domains, which are not equivalent. As explained earlier, some phishing domains are so successful or convincing that they themselves become popular domains. More robust or verifiable data could improve these results. And lastly, the 75th percentile threshold is a reasonable starting point, but could be refined via rigorous review or ROC-based threshold selection. Curves for the cosine similarity threshold or training on the raw embeddings vectors could also create better results.

VII. Conclusion and Potential Future Work

A. Overall Summary

In conclusion, our research successfully demonstrated a machine learning pipeline designed to predict whether a malicious URL is semantically linked to phishing or impersonation, facilitating the proactive remediation of a large proportion of malicious URLs. By extracting a robust lexical feature set (`path_depth`, `hyphen_count`, `digit_ratio`, etc.) and utilizing cosine similarity scores between Phishtank's known phishing URLs and the Tranco Top-1M legitimate domain dataset, we established a rigorous basis for automatic impersonation detection. The application of a 75th-percentile threshold on semantic similarity created a binary target variable of

mined semantic metadata, which, after additional cleaning, allowed a Support Vector Machine (SVM) with an RBF kernel to automatically classify domains of notability. This shows that our research question about a correlation between the 2 stands strong, with our overall goal of significantly prioritizing domains being more than possible. Evaluated through our above confusion matrices, ROC-AUC metrics, and linear SVM coefficients, our methodologies results also gave us data to prove that structural URL characteristics effectively capture impersonation intent. So ultimately, the integration of semantic and lexical analysis allowed for the proactive prioritization and mitigation of malicious indicators; our semantic similarity phase could reduce training data by 75%, allowing future lexical and semantic ML to reliably and effectively act upon the remaining data and learn from the semantic prioritization.

B. Significance

Three main areas are highlighted in terms of significant results. First, the data we captured shows that structural URL features carry measurable data for predicting whether a phishing URL closely mimics a legitimate domain in embedding space. Next, our literature and data sources noted that labeled Phishing data is hard to come by [2]. But, by using semantic reasoning to mine data, we are able to provide more resources to identify impersonation and malicious phishing activity. Embeddings and cosine similarity also have the benefit of being affordable to do on scale via Vector Databases, meaning that applying Vector DB paradigms to raw domain data can be seen to have benefits [15]. And after that, we were able to show that even a classic SVM classifier with hyperparameter tuning achieves meaningful discrimination between high-similarity ("impersonating") and low-similarity phishing URLs based solely on lexical URL structure. This shows that conventional ML processes can still provide meaningful benefits in the future.

C. Future Work

As this work moves into the future, a huge aspect could be incorporating additional feature construction: TLD categorization, URL brand keyword recognition, character n-gram frequencies, clustering results, or even Levenshtein distance to known legitimate domains. We could also compare the SVM against other classifiers (Random Forest, Gradient Boosting, MLP) to benchmark and likely even improve performance. After that, additional datasets could be used to augment and mitigate the biases of PhishTank and Tranco, as more data could allow more dimensions to arise and outliers to be found. And after extracting the linear SVM coefficients, we could use them to inform feature selection for the overall phishing detection pipeline, which could improve outcomes as well. The pipeline was additionally packaged as a lightweight URL triage tool in our repo, demonstrating real-world applicability by combining both models into a single interface (meeting our goals of human interface-ability). A natural next step for this triage tool could be incorporating a binary phishing/legitimate classifier as a first-pass filter. In its totality however, the research was a success, and showed that the often neglected base datum of a phishing domain is rich with semantic and ML potential for attack prevention.

URL	Impersonation	Verdict	Attack Pattern
google.com	49.0%	Likely Not Impersonating	Shortener / Redirector
googl.io	47.0%	Likely Not Impersonating	Shortener / Redirector
sites.google.com/view/paypal-secure-login/home	47.4%	Likely Not Impersonating	Token / Long URL Abuse
https://qrco.de/bfyD0b	61.9%	Likely Not Impersonating	Shortener / Redirector
https://docs.google.com/presentation/d/e/2PACX-1vS96MJIExf8/pub?start=false	3.6%	Likely Not Impersonating	Token / Long URL Abuse

TABLE II
INTERACTIVE USER TRIAGE TOOL EXAMPLE

REFERENCES

[1] Gabriela Brezeanu, Alexandru Archip, and Codrut-Georgian Artene. Phish fighter: Self updating machine learning shield against phishing kits based on HTML code analysis. 13:4460–4486.

[2] Cisco Systems, Inc. PhishTank: Join the fight against phishing. https://www.phishtank.com/, 2026.

[3] Valentin Danchev. Reproducible data science with python: An open learning resource. *Journal of Open Source Education*, 5:156, 10 2022.

[4] Bibhu Dash and Meraj Farheen Ansari. An effective cybersecurity awareness training model: First defense of an organizational security strategy. *International Research Journal of Engineering and Technology*, 9:2395–0056, 04 2022.

[5] DataCamp, Inc. DataCamp: Semantic Modeling and The Semantic Layer. https://www.datacamp.com/blog/semantic-layer, 2026.

[6] Dawn M. Sarno, Maggie W. Harris, and Jeffrey Black. Which phish is captured in the net? understanding phishing susceptibility and individual differences. 37(4):789–803. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/acp.4075.

[7] Giuseppe Desolda, Francesco Greco, and Luca Vigano. APOLLO: A GPT-based tool to detect phishing emails and generate explanations that warn users. 9(4):EICS003:1–EICS003:33.

[8] Manoel Herrera, Suzana Vilas, Silva , Pedro Antonio, Enzo Ferreira, Ricardo V Godoy, Leonardo André Ambrosio, and Marcelo Becker. The impact of feature scaling in machine learning: Effects on regression and classification tasks. *IEEE Access*, pages 1–1, 01 2025.

[9] Alphabet Inc. Textual Embeddings in the Gemini Vertex API. https://ai.google.dev/gemini-api/docs/embeddings, 2024.

[10] Inria. The 2019 inria-french academy of sciences-dassault systèmes innovation prize : scikit-learn , a success story for machine learning free software | inria, 01 2020.

[11] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*, 2019.

[12] Quanzhong Liu, Chihau Chen, Yang Zhang, and Zhengguo Hu. Feature selection for support vector machines with rbf kernel. *Artificial Intelligence Review*, 36:99–115, 02 2011.

[13] Nadjate Saidani, Kamel Adi, and Mohand Saïd Allili. A semantic-based classification approach for an enhanced spam detection. *Computers & Security*, 94, 2020-07.

[14] Sonowal and Gunikhan. Detecting phishing sms based on multiple correlation algorithms. *SN Computer Science*, 1, 11 2020.

[15] Yongye Su, Yinqi Sun, Minjia Zhang, and Jianguo Wang. Vexless: A serverless vector data management system using cloud functions. *Proceedings of the ACM on Management of Data*, 2:1–26, 05 2024.

[16] Panpan Zhang, Jing Ya, Tingwen Liu, Quangang Li, Jinqiao Shi, and Zhaojun Gu. imcircle: Automatic mining of indicators of compromise from the web. In *2019 IEEE Symposium on Computers and Communications (ISCC)*, 2019.

[17] Rasha Zieni, Luisa Massari, and Maria Carla Calzarossa. Phishing or not phishing? a survey on the detection of phishing websites. 11:18499–18519.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

APPENDIX

Note: We have the below code appendices in an interactive and visual Notebook format at https://github.com/ns15468-gasou/DS_ML_Project_ColabIntegration/tree/main/Project/notebooks

A. Code of Main Data Cleaning (Converted from .ipynb)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from urllib.parse import urlparse

import os

# Clone the repo (skip if already present)
if not os.path.exists('DS_ML_Project_ColabIntegration'):
    !git clone https://github.com/ns15468-gasou/DS_ML_Project_ColabIntegration.git

os.chdir('DS_ML_Project_ColabIntegration/Project/notebooks')
print("Working directory:", os.getcwd())

phish = pd.read_csv('../data/raw_input/verified_online.csv')
phish['target'] = phish['target'].str.replace('&amp;', '&', regex=False)
phish.head()

phish.shape

phish['submission_time'] = pd.to_datetime(phish['submission_time'])
phish['verification_time'] = pd.to_datetime(phish['verification_time'])
phish.dtypes

phish.isnull().sum()

(phish == '').sum()

target_counts = phish['target'].value_counts().head(15)
print(target_counts)

plt.figure(figsize=(12,6))
sns.barplot(x=target_counts.values, y=target_counts.index)
plt.title('Figure 1: Top 15 Phishing Targets')
plt.xlabel('Count')
plt.ylabel('Target')
plt.tight_layout()
plt.show()

target_counts_filtered = phish[phish['target'] != 'Other']['target'].value_counts().head(15)
plt.figure(figsize=(12,6))
sns.barplot(x=target_counts_filtered.values, y=target_counts_filtered.index)
plt.title('Figure 2: Top 15 Phishing Targets (excl. Other)')
plt.xlabel('Count')
plt.ylabel('Target')
plt.tight_layout()
plt.show()

online_counts = phish['online'].value_counts()
print(online_counts)

plt.figure(figsize=(6,4))
sns.barplot(x=online_counts.index, y=online_counts.values)
plt.title('Figure 3: Online vs Offline Phishing URLs')
plt.xlabel('Status')
plt.ylabel('Count')
plt.tight_layout()
plt.show()

phish['host'] = phish['url'].apply(lambda x: urlparse(x).netloc.lower())
phish['full_length'] = phish['url'].str.len()
phish['host_length'] = phish['host'].str.len()
phish['path'] = phish['url'].apply(lambda x: urlparse(x).path)
phish['path_depth'] = phish['path'].apply(lambda p: p.strip('/').count('/') + 1 if p.strip('/') else 0)
phish['has_path'] = (phish['path'].str.strip('/') != '').astype(int)
phish['tld_ext'] = phish['host'].str.rsplit('.', n=1).str[-1]
phish['dot_count'] = phish['url'].str.count(r'\.')
phish['hyphen_count'] = phish['url'].str.count('-')
phish['digit_count'] = phish['url'].str.count(r'\d')
phish['digit_ratio'] = phish['digit_count'] / phish['full_length']
phish['subdomain_depth'] = phish['host'].
```

```
    str.count(r'\.')

print("PhishTank - Raw Features")
phish[['full_length', 'host_length', '
    path_depth', 'dot_count', '
    hyphen_count',
     'digit_count', 'digit_ratio', '
        subdomain_depth']].describe()

clip_threshold = 0.98
phish_clipped = phish[['full_length', '
    host_length', 'path_depth', 'dot_count
    ', 'hyphen_count',
                'digit_count', '
                    digit_ratio', '
                    subdomain_depth']].
                    apply(
     lambda col: col.clip(upper=col.
        quantile(clip_threshold)))
print("PhishTank - Features Clipped at 98
    th Percentile")
phish_clipped.describe()

# URL length distribution (clipped to 98
    th percentile)
plt.figure(figsize=(10, 5))
sns.histplot(phish['full_length'], bins
    =80, edgecolor='white', alpha=0.85,
      binrange=(0, phish['full_length'].
          quantile(0.98)))
plt.axvline(phish['full_length'].mean(),
    ls='--', lw=1.2,
            label=f'Mean {phish["
                full_length"].mean():.1f}'
                )
plt.title('Figure 4: PhishTank URL Length
     Distribution')
plt.xlabel('Character Length')
plt.ylabel('Frequency')
plt.tight_layout()
plt.legend()
plt.show()

# Top 20 hosting domains
host_counts = phish['host'].value_counts
    ().head(20)

fig, ax = plt.subplots(figsize=(10, 6))
host_counts.plot.barh(ax=ax, edgecolor='
    white')
ax.set_title('Figure 5: PhishTank - Top
    20 Hosting Domains')
ax.set_xlabel('Number of Phishing URLs')
ax.invert_yaxis()

for i, v in enumerate(host_counts):
    ax.text(v + 1, i, str(v), va='center'
        , fontsize=9)

plt.tight_layout()
plt.show()

# Path depth distribution (clipped to 98
    th percentile)
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(phish['path_depth'],
        bins=range(0, int(phish['
            path_depth'].quantile(0.98)) +
             2),
        edgecolor='white', alpha=0.85)
ax.set_title('Figure 6: PhishTank - URL
    Path Depth Distribution (<=98th pctl)'
    )
ax.set_xlabel('Path Depth (number of /
    segments)')
ax.set_ylabel('Frequency')
plt.tight_layout()
plt.show()

# Structural complexity: dot count &
    subdomain depth
fig, axes = plt.subplots(1, 2, figsize
    =(13, 5))

axes[0].hist(phish['dot_count'], bins=
    range(0, phish['dot_count'].max() + 2)
    ,
      edgecolor='white', alpha=0.85)
axes[0].set_title('Figure 7a: Dot Count
    Distribution (full URL)')
axes[0].set_xlabel('Number of dots')
axes[0].set_ylabel('Frequency')

axes[1].hist(phish['subdomain_depth'],
    bins=range(0, phish['subdomain_depth'
    ].max() + 2),
      edgecolor='white', alpha=0.85)
axes[1].set_title('Figure 7b: Subdomain
    Depth (dots in hostname)')
axes[1].set_xlabel('Subdomain depth')
axes[1].set_ylabel('Frequency')

fig.suptitle('Structural Complexity -
    PhishTank URLs', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

# Character composition: hyphens & digit
    ratio
fig, axes = plt.subplots(1, 2, figsize
    =(13, 5))
```

```
axes[0].hist(phish['hyphen_count'], bins=
    range(0, 10),
      edgecolor='white', alpha=0.85)
axes[0].set_title('Figure 8a: Hyphen
    Count Distribution')
axes[0].set_xlabel('Number of hyphens')
axes[0].set_ylabel('Frequency')

axes[1].hist(phish['digit_ratio'], bins
    =50,
       edgecolor='white', alpha=0.85,
           range=(0, 0.5))
axes[1].set_title('Figure 8b: Digit Ratio
     Distribution')
axes[1].set_xlabel('Proportion of digits
    in URL')
axes[1].set_ylabel('Frequency')

fig.suptitle('Character Composition -
    PhishTank URLs', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

tranco = pd.read_csv('../data/raw_input/
    top-1m.csv', header=None, names=['rank
    ', 'domain'])
tranco.head()

tranco.shape

tranco.dtypes

tranco.isnull().sum()

(tranco == '').sum()

duplicateDomains = tranco['domain'].
    duplicated().sum()
duplicateRanks = tranco['rank'].
    duplicated().sum()
print(f'Duplicate domains: {
    duplicateDomains}')
print(f'Duplicate ranks: {duplicateRanks}
    ')

tranco['tld'] = tranco['domain'].str.
    split('.').str[-1]
tld_counts = tranco['tld'].value_counts()
    .head(15)
print(tld_counts)

plt.figure(figsize=(12,6))
sns.barplot(x=tld_counts.values, y=
    tld_counts.index)
plt.title('Figure 9: Top 15 TLDs in
    Tranco Top 1M')
plt.xlabel('Count')
plt.ylabel('TLD')
plt.tight_layout()
plt.show()

tranco['domain_length'] = tranco['domain'
    ].str.len()
print(tranco['domain_length'].describe())

plt.figure(figsize=(10, 5))
sns.histplot(tranco['domain_length'],
    bins=50, kde=True)
plt.title('Figure 10: Domain Length
    Distribution (Tranco Top 1M)')
plt.xlabel('Character Length')
plt.ylabel('Count')
plt.tight_layout()
plt.show()

tranco['dot_count'] = tranco['domain'].
    str.count(r'\.')
tranco['hyphen_count'] = tranco['domain'
    ].str.count('-')
tranco['digit_count'] = tranco['domain'].
    str.count(r'\d')
tranco['digit_ratio'] = tranco['
    digit_count'] / tranco['domain_length'
    ]
tranco['subdomain_depth'] = tranco['
    domain'].str.count(r'\.')

tranco[['domain_length', 'dot_count', '
    hyphen_count', 'digit_count', '
    digit_ratio', 'subdomain_depth']].
    describe()

fig, axes = plt.subplots(1, 2, figsize
    =(13, 5))

bins_dots = range(0, tranco['dot_count'].
    max() + 2)
axes[0].hist(tranco['dot_count'], bins=
    bins_dots, color='steelblue',
    edgecolor='white', alpha=0.85)
axes[0].set_title('Figure 11a: Dot Count
    Distribution')
axes[0].set_xlabel('Number of dots')
axes[0].set_ylabel('Frequency')

bins_sub = range(0, tranco['
    subdomain_depth'].max() + 2)
axes[1].hist(tranco['subdomain_depth'],
    bins=bins_sub, color='steelblue',
    edgecolor='white', alpha=0.85)
axes[1].set_title('Figure 11b: Subdomain
    Depth (dots in domain)')
```

```
axes[1].set_xlabel('Subdomain depth')
axes[1].set_ylabel('Frequency')

fig.suptitle('Structural Complexity -
    Tranco Domains', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize
    =(13, 5))

axes[0].hist(tranco['hyphen_count'], bins
    =range(0, 10), color='steelblue',
    edgecolor='white', alpha=0.85)
axes[0].set_title('Figure 12a: Hyphen
    Count Distribution')
axes[0].set_xlabel('Number of hyphens')
axes[0].set_ylabel('Frequency')

axes[1].hist(tranco['digit_ratio'], bins
    =50, color='steelblue', edgecolor='
    white', alpha=0.85)
axes[1].set_title('Figure 12b: Digit
    Ratio Distribution')
axes[1].set_xlabel('Proportion of digits
    in domain')
axes[1].set_ylabel('Frequency')

fig.suptitle('Character Composition -
    Tranco Domains', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

phish['domain'] = phish['url'].apply(
    lambda x:
     urlparse(x).netloc.lower().replace('
        www.', '', 1))

tranco_domains = set(tranco['domain'].str
    .lower())
overlap = phish['domain'].isin(
    tranco_domains).sum()
print(f'Phishing URLs whose domain
    appears in Tranco Top 1M: {overlap} /
    {len(phish)} ({overlap/len(phish)
    *100:.2f}%)')

print(f"Tranco: {len(tranco):,}
    legitimate domains (ranked by
    popularity)")
print(f"PhishTank: {len(phish):,}
    verified phishing URLs")
print(f"Overlap: {overlap:,} phishing
    domains found in Tranco top 1M ({
    overlap/len(phish)*100:.2f}%)")
print("\nNotes:\n- Tranco aggregates
    multiple blocklists/popularity sources
    ; high rank != guaranteed safe\n-
    PhishTank is a snapshot of verified-
    online phishing URLs, not exhaustive\n
    - Overlap may reflect compromised
    legitimate domains or shared hosting
    infrastructure")

summary_data = {
    "Metric": [
        "Count", "Mean length", "Median
            length", "Max length",
        "Mean dots", "Mean hyphens", "
            Mean digit ratio",
        "Unique TLDs", "Mean subdomain
            depth"
    ],
    "Tranco (domain)": [
        f"{len(tranco):,}",
        f"{tranco['domain_length'].mean()
            :.1f}",
        f"{tranco['domain_length'].median
            ():.0f}",
        f"{tranco['domain_length'].max()}
            ",
        f"{tranco['dot_count'].mean():.2f
            }",
        f"{tranco['hyphen_count'].mean()
            :.2f}",
        f"{tranco['digit_ratio'].mean()
            :.4f}",
        f"{tranco['tld'].nunique()}",
        f"{tranco['subdomain_depth'].mean
            ():.2f}",
    ],
    "PhishTank (full URL)": [
        f"{len(phish):,}",
        f"{phish['full_length'].mean():.1
            f}",
        f"{phish['full_length'].median()
            :.0f}",
        f"{phish['full_length'].max()}",
        f"{phish['dot_count'].mean():.2f}
            ",
        f"{phish['hyphen_count'].mean()
            :.2f}",
        f"{phish['digit_ratio'].mean():.4
            f}",
        f"{phish['tld_ext'].nunique()}",
        f"{phish['subdomain_depth'].mean
            ():.2f}",
    ],
}
summary_tbl = pd.DataFrame(summary_data)

fig, ax = plt.subplots(figsize=(10, 4))
```

```
ax.axis("off")
tbl = ax.table(cellText=summary_tbl.
    values, colLabels=summary_tbl.columns,
        loc="center", cellLoc="center")
tbl.auto_set_font_size(False)
tbl.set_fontsize(10)
tbl.scale(1.2, 1.6)
for (row, col), cell in tbl.get_celld().
    items(): # From Datacamp [9],
    alternating row colors and header
    styling
     if row == 0:
         cell.set_facecolor("#4472C4")
         cell.set_text_props(color="white"
             , fontweight="bold")
     elif row % 2 == 0:
         cell.set_facecolor("#D9E2F3")
ax.set_title("Figure 12: Side-by-Side
    Comparison: Tranco vs PhishTank",
    fontsize=13,
      fontweight="bold", pad=20)
plt.tight_layout()
plt.show()

phish_num = phish[['full_length', '
    host_length', 'path_depth', 'dot_count
    ',
            'hyphen_count', 'digit_count',
                'digit_ratio', '
                subdomain_depth']]
tranco_num = tranco[['domain_length', '
    dot_count', 'hyphen_count',
              'digit_count', 'digit_ratio'
                  , 'subdomain_depth']]

fig, axes = plt.subplots(1, 2, figsize
    =(18, 7))

# PhishTank heatmap
corr_phish = phish_num.corr()
sns.heatmap(corr_phish, annot=True, fmt='
    .2f', cmap='coolwarm', center=0,
              linewidths=0.5, ax=axes[0],
                  vmin=-1, vmax=1)
axes[0].set_title('Figure 13a: PhishTank
    - Feature Correlation', fontsize=12)

# Tranco heatmap
corr_tranco = tranco_num.corr()
sns.heatmap(corr_tranco, annot=True, fmt=
    '.2f', cmap='coolwarm', center=0,
              linewidths=0.5, ax=axes[1],
                  vmin=-1, vmax=1)
axes[1].set_title('Figure 13b: Tranco -
    Feature Correlation', fontsize=12)
plt.tight_layout()
plt.show()

# Interpret strongest correlations for
    each dataset
for name, corr in [('PhishTank',
    corr_phish), ('Tranco', corr_tranco)]:
    # Upper triangle only (no duplicates
        / diagonal)
    mask = pd.np.triu(pd.np.ones(corr.
        shape), k=1).astype(bool) if
        hasattr(pd, 'np') else __import__(
        'numpy').triu(__import__('numpy').
        ones(corr.shape), k=1).astype(bool
        )
    pairs = corr.where(mask).stack().
        reset_index()
    pairs.columns = ['Feature A', '
        Feature B', 'r']
    pairs['abs_r'] = pairs['r'].abs()
    top = pairs.nlargest(3, 'abs_r')
    print(f"\n{name} - Top 3 strongest
        correlations:")
    for _, row in top.iterrows():
        print(f"  {row['Feature A']} <->
            {row['Feature B']}: r = {row['
            r']:.3f}")

import numpy as np

# Build a combined dataframe with a '
    source' label as a target variable
phish_sample = phish[['host_length', '
    dot_count', 'hyphen_count', '
    digit_ratio', 'subdomain_depth']].copy
    ()
phish_sample['source'] = 'PhishTank'

tranco_sample = tranco[['domain_length',
    'dot_count', 'hyphen_count', '
    digit_ratio', 'subdomain_depth']].copy
    ()
tranco_sample = tranco_sample.rename(
    columns={'domain_length': 'host_length
    '})
tranco_sample['source'] = 'Tranco'

combined = pd.concat([phish_sample,
    tranco_sample], ignore_index=True)

# Pair plot colored by dataset source
g = sns.pairplot(combined.sample(n=min
    (4000, len(combined)), random_state
    =42),
          hue='source', vars=['host_length
              ', 'dot_count', 'hyphen_count
```

```
            ', 'digit_ratio'],
        diag_kind='hist', plot_kws={'
            alpha': 0.4, 's': 15},
        palette={'PhishTank': 'salmon',
            'Tranco': 'steelblue'})
g.figure.suptitle('Figure 14: Pair Plot -
    PhishTank vs Tranco Structural
    Features', y=1.02, fontsize=14)
# Adjust the layout so the legend isn't
    cut off (instead of tight_layout)
sns.move_legend(g, "upper left",
    bbox_to_anchor=(1, 0.5), title='Source
    ')
plt.subplots_adjust(right=0.85, top=0.95)
plt.show()

from sklearn.preprocessing import
    StandardScaler

scale_features = ['full_length', '
    host_length', 'path_depth', 'dot_count
    ',
          'hyphen_count', 'digit_ratio',
            'subdomain_depth']

scaler = StandardScaler()
phish_scaled = pd.DataFrame(scaler.
    fit_transform(phish[scale_features]),
    columns=[f'{c}_scaled' for c in
    scale_features])

phish_scaled.describe().round(2)

phish.to_csv('../data/processed_input/
    phishtank_cleaned.csv', index=False)
tranco.to_csv('../data/processed_input/
    tranco_cleaned.csv', index=False)
print("Exported phishtank_cleaned.csv and
    tranco_cleaned.csv.")

# standard-library I/O
import csv
import json
import sys
import gc

# use min-heap for priority queue when
    choosing top-N matches
import heapq

# safe path handling across ipynb and
    Google resources
from pathlib import Path

# cosine similarity and other vector
    operations
import numpy as np

# used to easily export to CSV and handle
    dataframes
import pandas as pd

BASE_DIR = Path.cwd().parent  # one level
    up from notebook cwd
DATA_DIR = BASE_DIR / "data"  # ../data

# Part A I/O
EMBEDS1_FILE = DATA_DIR / "embeds1.csv"
EMBEDS2_FILE = DATA_DIR / "embeds2.csv"
EMBEDSA_FILE = DATA_DIR / "embedsA.csv"
OUT_MATRIX   = DATA_DIR / "
    FullMatrixOfCosineSimil.csv"

# Part B I/O
FULL_MATRIX_FILE = DATA_DIR / "
    FullMatrixOfCosineSimil.csv"
OUT_ROW          = DATA_DIR / "
    top5_per_row.csv"
OUT_COL          = DATA_DIR / "
    top5_per_column.csv"

TOP_N = 5 # Used if we wanted to test
    more or fewer top matches

import urllib.request
import shutil

_GCS_BASE = "https://storage.googleapis.
    com/gscsteam1-colabresultbucket"
_EMBED_FILES = {
    EMBEDS1_FILE: f"{_GCS_BASE}/embeds1.
        csv",
    EMBEDS2_FILE: f"{_GCS_BASE}/embeds2.
        csv",
    EMBEDSA_FILE: f"{_GCS_BASE}/embedsA.
        csv",
}

for local_path, url in _EMBED_FILES.items
    ():
    if local_path.exists():
        print(f"  Already exists,
            skipping: {local_path.name}")
        continue
    print(f"  Downloading {local_path.
        name} ...", flush=True)
    local_path.parent.mkdir(parents=True,
        exist_ok=True)
    with urllib.request.urlopen(url) as
        resp, open(local_path, "wb") as
        out:
        shutil.copyfileobj(resp, out,
```

```
            length=8 * 1024 * 1024)  # 8
            MB chunks
        size_gb = local_path.stat().st_size /
            1e9
        print(f"    Done ({size_gb:.2f} GB)")

print("All embedding files ready.")

def load_embeddings(filepath: Path) ->
    dict[str, np.ndarray]:

    result = {}
    with open(filepath, "r", encoding="
        utf-8", newline="") as f:
        reader = csv.reader(f)
        next(reader, None)  # skip header
            row (embedding,text)
        for row in reader:
            if len(row) < 2:
        continue
            embedding_str, name = row[0],
                row[1]
            if name not in result:  #
                keep first occurrence
        result[name] = np.array(json.
            loads(embedding_str), dtype=np
            .float32)
    return result

def build_matrix(app_dict: dict[str, list
    [float]]) -> tuple[list[str], np.
    ndarray]:

    names = list(app_dict.keys())
    mat = np.array([app_dict[n] for n in
        names], dtype=np.float32)
    return names, mat

def cosine_similarity_matrix(A: np.
    ndarray, B: np.ndarray) -> np.ndarray:

    A_norm = A / np.linalg.norm(A, axis
        =1, keepdims=True)
    B_norm = B / np.linalg.norm(B, axis
        =1, keepdims=True)
    return A_norm @ B_norm.T

print("Loading embeddings files")
emb1 = load_embeddings(EMBEDS1_FILE)
emb2 = load_embeddings(EMBEDS2_FILE)

# Merge: emb1 takes precedence; add only
    new keys from emb2
combined = {}
for name, vec in emb1.items():
    combined[name] = vec
for name, vec in emb2.items():
    if name not in combined:
        combined[name] = vec
del emb1, emb2  # free memory
gc.collect()

print(f"  Combined & deduplicated: {len(
    combined)} unique entries\n")

embA = load_embeddings(EMBEDSA_FILE)

import re

def clean_url(url: str) -> str:

    url = re.sub(r'^https?://', '', url)
    url = re.sub(r'^www\.', '', url)
    url = url.rstrip('/')
    return url

def deduplicate_cleaned(emb_dict: dict[
    str, np.ndarray]) -> tuple[dict[str,
    np.ndarray], pd.DataFrame]:

    cleaned: dict[str, np.ndarray] = {}
    mapping: list[dict[str, str]] = []
    for raw_key, vec in emb_dict.items():
        key = clean_url(raw_key)
        mapping.append({"url": raw_key, "
            cleaned_url": key})
        if key not in cleaned:
            cleaned[key] = vec
    return cleaned, pd.DataFrame(mapping)

before_combined = len(combined)
combined, map_combined =
    deduplicate_cleaned(combined)
print(f"  combined: {before_combined} ->
    {len(combined)} after URL cleaning")

before_A = len(embA)
embA, map_embA = deduplicate_cleaned(embA
    )
print(f"  embA:     {before_A} -> {len(
    embA)} after URL cleaning")

# Export URL mappings (url column is
    unique; cleaned_url may have
    duplicates)
OUT_MAP_COMBINED = DATA_DIR / "
    url_cleaning_map_phishtank.csv"
OUT_MAP_EMBA     = DATA_DIR / "
    url_cleaning_map_tranco.csv"
map_combined.to_csv(OUT_MAP_COMBINED,
    index=False)
map_embA.to_csv(OUT_MAP_EMBA, index=False
```

```
    )
print(f"  Saved {OUT_MAP_COMBINED.name}
    ({len(map_combined)} rows)")
print(f"  Saved {OUT_MAP_EMBA.name}  ({
    len(map_embA)} rows)")

del map_combined, map_embA
gc.collect()

print("Building numpy arrays")
names_12, mat_12 = build_matrix(combined)
del combined
gc.collect()

names_A, mat_A = build_matrix(embA)
del embA
gc.collect()

print(f"  Matrix 1&2: {mat_12.shape}  |
    Matrix A: {mat_A.shape}")
mem_mb = (mat_12.nbytes + mat_A.nbytes) /
     1e6
print(f"  Embedding memory: {mem_mb:.1f}
    MB")

CHUNK_SIZE = 500  # rows per chunk; keeps
     each chunk < 200 MB for 12 GB Colab

# L2-normalise in-place so we only need
    the dot product for cosine sim
norms_12 = np.linalg.norm(mat_12, axis=1,
     keepdims=True)
norms_12[norms_12 == 0] = 1.0
mat_12 /= norms_12
del norms_12

norms_A = np.linalg.norm(mat_A, axis=1,
    keepdims=True)
norms_A[norms_A == 0] = 1.0
mat_A /= norms_A
del norms_A
gc.collect()

n_rows = mat_12.shape[0]
print(f"Computing cosine similarity in
    {-(-n_rows // CHUNK_SIZE)} chunks "
       f"& streaming to CSV ...", flush=
           True)

with open(OUT_MATRIX, "w", encoding="utf
    -8", newline="") as f:
     writer = csv.writer(f)
     writer.writerow(["App_Embeds12"] +
         names_A)

     for start in range(0, n_rows,
         CHUNK_SIZE):
      end = min(start + CHUNK_SIZE,
          n_rows)
      chunk_sim = mat_12[start:end] @
          mat_A.T          # (chunk, M)
          float32
      for i, row_idx in enumerate(range
          (start, end)):
       writer.writerow(
      [names_12[row_idx]] + [f"{v:.6f}"
           for v in chunk_sim[i]]
       )
      del chunk_sim
      gc.collect()
      if start == 0 or (start //
          CHUNK_SIZE) % 20 == 0:
       print(f"  {end:,} / {n_rows
           :,} rows written", flush=
           True)

print(f"  Done - saved {OUT_MATRIX.name}"
    )

size_gb = OUT_MATRIX.stat().st_size / 1e9
print(f"  File:  {OUT_MATRIX}")
print(f"  Size:  {size_gb:.2f} GB")
print(f"  Shape: ({len(names_12)}, {len(
    names_A)})")

# Free normalised matrices now that the
    CSV is written
del mat_12, mat_A
gc.collect()

print("Reading header ...", flush=True)
with open(FULL_MATRIX_FILE, "r", encoding
    ="utf-8", newline="") as f:
     reader = csv.reader(f)
     header = next(reader)

# header[0] is the corner label (e.g. "
    App_Embeds12"), rest are column names
col_labels = header[1:]
num_cols = len(col_labels)
print(f"  {num_cols} columns detected.")

# Min-heaps for each column: heap of (
    similarity, row_label)
col_heaps: list[list[tuple[float, str]]]
    = [[] for _ in range(num_cols)]

print("Streaming rows ...", flush=True)
row_results: list[tuple[str, list[tuple[
    float, str]]]] = []
row_count = 0
```

```
with open(FULL_MATRIX_FILE, "r", encoding
    ="utf-8", newline="") as f:
    reader = csv.reader(f)
    next(reader)  # skip header

    for row in reader:
        row_label = row[0]
        values = row[1:]

        # --- top 5 for this row (min-
            heap of size TOP_N) ---
        row_pairs: list[tuple[float, str
            ]] = []
        for j, val_str in enumerate(
            values):
            if j >= num_cols:
        break
            try:
        sim_val = float(val_str)
            except (ValueError,
                IndexError):
        continue

            # Row top-5
            if len(row_pairs) < TOP_N:
        heapq.heappush(row_pairs, (
            sim_val, col_labels[j]))
            elif sim_val > row_pairs
                [0][0]:
        heapq.heapreplace(row_pairs, (
            sim_val, col_labels[j]))

            # Column top-5
            heap = col_heaps[j]
            if len(heap) < TOP_N:
        heapq.heappush(heap, (sim_val,
            row_label))
            elif sim_val > heap[0][0]:
        heapq.heapreplace(heap, (sim_val,
            row_label))

        # Store row result sorted
            descending
        row_pairs.sort(reverse=True)
        row_results.append((row_label,
            row_pairs))

        row_count += 1
        if row_count % 5000 == 0:
            print(f"  {row_count:,} rows
                processed", flush=True)

print(f"  Done: {row_count:,} rows")

print(f"Writing {OUT_ROW.name} ...",
    flush=True)
with open(OUT_ROW, "w", encoding="utf-8",
    newline="") as f:
    writer = csv.writer(f)
    writer.writerow(
        ["item"]
        + [f"match_{i+1}" for i in range(
            TOP_N)]
        + [f"score_{i+1}" for i in range(
            TOP_N)]
    )
    for row_label, pairs in row_results:
        names = [p[1] for p in pairs]
        scores = [f"{p[0]:.6f}" for p in
            pairs]
        while len(names) < TOP_N:
            names.append("")
            scores.append("")
        writer.writerow([row_label] +
            names + scores)

print(f"  Saved: {OUT_ROW}")

print(f"Writing {OUT_COL.name} ...",
    flush=True)
with open(OUT_COL, "w", encoding="utf-8",
    newline="") as f:
    writer = csv.writer(f)
    writer.writerow(
        ["item"]
        + [f"match_{i+1}" for i in range(
            TOP_N)]
        + [f"score_{i+1}" for i in range(
            TOP_N)]
    )
    for j, col_label in enumerate(
        col_labels):
        pairs = sorted(col_heaps[j],
            reverse=True)
        names = [p[1] for p in pairs]
        scores = [f"{p[0]:.6f}" for p in
            pairs]
        while len(names) < TOP_N:
            names.append("")
            scores.append("")
        writer.writerow([col_label] +
            names + scores)

print(f"  Saved: {OUT_COL}")

print("Part B complete")
print(f"  Rows streamed:  {row_count:,}")
print(f"  Columns:        {num_cols:,}")
print(f"\nOutputs:")
print(f"  -> {OUT_ROW}")
print(f"  -> {OUT_COL}")
```

```
# Load the top-5 similarity results and
    reference datasets
df_top = pd.read_csv(OUT_COL)    # Top
    Domains compared to Phishes
df_phish = pd.read_csv(OUT_ROW) # Phishes
     compared to Top Domains

phishtank_urls = pd.read_csv(DATA_DIR / "
    processed_input" / "phishtank_cleaned.
    csv")
tranco_domains = pd.read_csv(DATA_DIR / "
    processed_input" / "tranco_cleaned.csv
    ")

score_cols = [c for c in df_top.columns
    if c.startswith("score_")]
match_cols = [c for c in df_top.columns
    if c.startswith("match_")]

print(f"Top Domains -> Phishes : {df_top.
    shape}")
print(f"Phishes -> Top Domains : {
    df_phish.shape}")
print(f"PhishTank URLs         : {
    phishtank_urls.shape}")
print(f"Tranco Domains         : {
    tranco_domains.shape}")

print("Top Domains -> Phish Matches (
    scores)")
print(df_top[score_cols].describe().
    to_string())
print()
print("Phish -> Top Domain Matches (
    scores)")
print(df_phish[score_cols].describe().
    to_string())

print("TOP 10 HIGHEST score_1 - Top
    Domains -> Phish Matches")
print(df_top.nlargest(10, "score_1")[["
    item", "match_1", "score_1"]].
    to_string(index=False))
print()
print("TOP 10 HIGHEST score_1 - Phish ->
    Top Domain Matches")
print(df_phish.nlargest(10, "score_1")[["
    item", "match_1", "score_1"]].
    to_string(index=False))
print()
print("BOTTOM 10 LOWEST score_1 - Top
    Domains -> Phish Matches") # We use
    the below maps to limit the strings to
     15 chars for better readability
print(df_top.nsmallest(10, "score_1")[["
    item", "match_1", "score_1"]].applymap
    (lambda x: str(x)[:15] if isinstance(x
    , str) else x).to_string(index=False))
print()
print("BOTTOM 10 LOWEST score_1 - Phish
    -> Top Domain Matches")
print(df_phish.nsmallest(10, "score_1")[[
    "item", "match_1", "score_1"]].
    applymap(lambda x: str(x)[:15] if
    isinstance(x, str) else x).to_string(
    index=False))

means_top = df_top[score_cols].mean()
means_phish = df_phish[score_cols].mean()

print("Top->Phish:  " + "  ".join(f"{c}={
    v:.4f}" for c, v in means_top.items())
    )
print("Phish->Top:  " + "  ".join(f"{c}={
    v:.4f}" for c, v in means_phish.items
    ()))
print(f"Top->Phish  avg drop (1->5): {
    means_top['score_1'] - means_top['
    score_5']:.4f}")
print(f"Phish->Top  avg drop (1->5): {
    means_phish['score_1'] - means_phish['
    score_5']:.4f}")

fig, axes = plt.subplots(1, 2, figsize
    =(12, 5), sharey=True)
axes[0].hist(df_top["score_1"], bins=50,
    edgecolor="white", alpha=0.85)
axes[0].set_title("Top Domains -> Closest
     Phish\n(score_1 distribution)")
axes[0].set_xlabel("Cosine Similarity (
    score_1)")
axes[0].set_ylabel("Frequency")
axes[1].hist(df_phish["score_1"], bins
    =50, edgecolor="white", alpha=0.85)
axes[1].set_title("Phish URLs -> Closest
    Legit Domain\n(score_1 distribution)")
axes[1].set_xlabel("Cosine Similarity (
    score_1)")
fig.suptitle("Figure 15 - Distribution of
     Best-Match Cosine Similarity",
    fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize
    =(12, 5))
axes[0].boxplot([df_top[c].dropna() for c
     in score_cols],
        tick_labels=[f"Match {i+1}" for i
            in range(5)])
axes[0].set_title("Top Domains -> Phish
    Matches")
```

```
axes[0].set_ylabel("Cosine Similarity")
axes[1].boxplot([df_phish[c].dropna() for
    c in score_cols],
        tick_labels=[f"Match {i+1}" for i
            in range(5)])
axes[1].set_title("Phish -> Top Domain
    Matches")
axes[1].set_ylabel("Cosine Similarity")
fig.suptitle("Figure 16 - Score
    Distribution Across Top-5 Matches",
    fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(7, 5))
x = range(1, 6)
ax.plot(x, means_top.values, "o-", label=
    "Top Domains -> Phish", linewidth=2)
ax.plot(x, means_phish.values, "s-",
    label="Phish -> Top Domains",
    linewidth=2)
ax.set_xticks(list(x))
ax.set_xticklabels([f"Match {i}" for i in
    x])
ax.set_ylabel("Mean Cosine Similarity")
ax.set_title("Figure 17 - Average Score
    Drop-off Across Top-5 Matches")
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# Remove exact overlaps between Tranco-
    side items and Phish-side items
top_items = set(df_top["item"].astype(str
    ).str.lower())
phish_items = set(df_phish["item"].astype
    (str).str.lower())
exact_overlap = top_items & phish_items

# Exclude rows where item or any matched
    domain entry is an exact overlap
mask_overlap = (
    df_phish["item"].astype(str).str.
        lower().isin(exact_overlap) |
    df_phish[match_cols].apply(lambda c:
        c.astype(str).str.lower().isin(
        exact_overlap)).any(axis=1)
)

# Top 20 by strongest nearest-neighbor
    similarity
top20p = df_phish.loc[~mask_overlap].
    nlargest(20, "score_1").copy()

# Compact similarity vector for easy
    scanning
top20p["similarity_vector"] = top20p[
    score_cols].apply(
    lambda r: "[" + ", ".join(f"{v:.3f}"
        for v in r.values) + "]", axis=1
)

# Final sorted table
out_tbl = top20p[
    ["item", "match_1", "score_1"]
].rename(columns={"match_1": "
    closest_real_url"}).sort_values("
    score_1", ascending=False)

out_tbl = out_tbl.reset_index(drop=True)
out_tbl.index = out_tbl.index + 1

print("Figure 18 - Top 20 Phish URLs Most
    Similar to a Legitimate Domain (table
    view)")
fig, ax = plt.subplots(figsize=(10, 8))
ax.axis('off')
tbl = ax.table(cellText=out_tbl.values,
    colLabels=out_tbl.columns, loc='center
    ', cellLoc='left')

# consistent styling from earlier
for (row, col), cell in tbl.get_celld().
    items():
    if row == 0:
        cell.set_facecolor("#4472C4")
        cell.set_text_props(color="white"
            , fontweight="bold")
    elif row % 2 == 0:
        cell.set_facecolor("#D9E2F3")

plt.show()

fig, axes = plt.subplots(1, 2, figsize
    =(12, 5))
axes[0].scatter(df_top["score_1"], df_top
    ["score_5"], alpha=0.3, s=8)
axes[0].plot([0.5, 1], [0.5, 1], "k--",
    alpha=0.3)
axes[0].set_xlabel("score_1")
axes[0].set_ylabel("score_5")
axes[0].set_title("Top Domains -> Phish\
    nscore_1 vs score_5")

axes[1].scatter(df_phish["score_1"],
    df_phish["score_5"], alpha=0.15, s=5)
axes[1].plot([0.5, 1], [0.5, 1], "k--",
    alpha=0.3)
axes[1].set_xlabel("score_1")
axes[1].set_ylabel("score_5")
axes[1].set_title("Phish -> Top Domains\
```

```
    nscore_1 vs score_5")

fig.suptitle("Figure 19 - Spread Between
    Best and 5th-Best Match", fontsize=14,
     y=1.02)
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(8, 5))
for label, series in [("Top->Phish",
    df_top["score_1"]),
                ("Phish->Top", df_phish["
                    score_1"])]:
     sorted_s = np.sort(series.dropna())
     cdf = np.arange(1, len(sorted_s) + 1)
          / len(sorted_s)
     ax.plot(sorted_s, cdf, label=label,
         linewidth=1.5)
ax.set_xlabel("Cosine Similarity (score_1
    )")
ax.set_ylabel("Cumulative Proportion")
ax.set_title("Figure 20 - CDF of Best-
    Match Cosine Similarity")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

B. Code of Main Data Embedding (Converted from .ipynb)

```
from google.colab import auth
auth.authenticate_user()

import subprocess
token = subprocess.check_output(
    ["gcloud", "auth", "print-identity-
        token"],
     text=True
).strip()

print("Authentication successful.")

import requests
import pandas as pd
import time
import os

INPUT_FILE = "phishtank_cleaned_minimal.
    txt"
OUTPUT_FILE = "local_embeddings_results.
    csv"
FUNCTION_URL = "https://embed-text-files-
    XXXXXXXXXXX.us-east1.run.app" #
    Redacted to prevent high costs on a
    shared notebook
BATCH_SIZE = 200 # The max the Gemini
    Embeddings API can handle in one
    request

# Below is the code of the embeddings
    cloud run function. Having it as a
    function rather than running locally
    lets it scale, authenticate, and cross
     communicate easily.

with open(INPUT_FILE, 'r') as f:
    lines = [line.strip() for line in f
        if line.strip()]

print(f"Total lines to process: {len(
    lines)}")

for i in range(0, len(lines), BATCH_SIZE)
    :
    chunk = lines[i : i + BATCH_SIZE]

    payload = {"texts": chunk}

    try:
        headers = {
            "Authorization": f"Bearer {
                token}",
            "Content-Type": "application/
                json",
        }
        response = requests.post(
            FUNCTION_URL, json=payload,
            headers=headers)
        response.raise_for_status()
        data = response.json()

        df_batch = pd.DataFrame(data['
            results'])

        # Write header only on the first
            batch
        write_header = not os.path.exists
            (OUTPUT_FILE)
        df_batch.to_csv(OUTPUT_FILE, mode
            ='a', index=False, header=
            write_header)

        print(f"Successfully processed
            batch {i // BATCH_SIZE + 1} ({
            i + len(chunk)}/{len(lines)})"
            )

    except Exception as e:
        print(f"Error at batch starting
            at index {i}: {e}")
        time.sleep(2)
```

```
if os.path.exists(OUTPUT_FILE):
    df = pd.read_csv(OUTPUT_FILE)
    print(f"Output saved to {OUTPUT_FILE}
        - {len(df)} rows, {len(df.columns
        )} columns")
    df.head()
```

C. Code of SVM Model (Converted from .ipynb)

```
# System, file, & URL utilities for
    handling OS paths, regex patterns, and
    parsing URLs for data ingestion
import os
import re
from pathlib import Path
from urllib.parse import urlparse

# Data Manipulation for handling
    DataFrames and vector/matrix math
import pandas as pd
import numpy as np

# For Exploratory Data Analysis (EDA) and
    making sense of distributions
import matplotlib.pyplot as plt
import seaborn as sns

# Model Selection & Cross-Validation for
    splitting data into sets and searching
    for the best hyperparameters
from sklearn.model_selection import
    train_test_split, RandomizedSearchCV,
    StratifiedKFold

# Preprocessing & Feature Scaling for
    ensuring features are on a similar
    scaling for ML
from sklearn.preprocessing import
    StandardScaler

# Our primary ML model for the
    classification task
from sklearn.svm import SVC

# Meeting the best practice of pipelining
    our ML model
from sklearn.pipeline import Pipeline

# And finally, some imports for
    quantifying how well the model
    performed through reports and visual
    curves
from sklearn.metrics import (
    classification_report,
        confusion_matrix,
        ConfusionMatrixDisplay,
    roc_curve, auc, RocCurveDisplay
)

# Clone the repo (skip if already present
    )
if not os.path.exists('
    DS_ML_Project_ColabIntegration'):
    !git clone https://github.com/ns15468
        -gasou/
        DS_ML_Project_ColabIntegration.git

os.chdir('DS_ML_Project_ColabIntegration/
    Project/notebooks')
print("Working directory:", os.getcwd())

BASE_DIR = Path.cwd().parent
DATA_DIR = BASE_DIR / "data"

# Input files
TOP5_PHISH_FILE = DATA_DIR / "
    summarized_output" / "top5_per_phish.
    csv"
PHISHTANK_FILE  = DATA_DIR / "raw_input"
    / "verified_online.csv"
URL_MAP_FILE    = DATA_DIR / "
    processed_input" / "
    url_cleaning_map_phishtank.csv"

# SVM settings (these are basic setups
    after looking at best practices from
    inclass and Datacamp)
RANDOM_STATE = 42
TEST_SIZE    = 0.2

def load_data(filepath):

    df = pd.read_csv(filepath)
    print(f"Loaded {len(df):,} phishing
        URLs with similarity scores")
    print(f"score_1 range: [{df['score_1
        '].min():.4f}, {df['score_1'].max
        ():.4f}]")
    return df

def load_cleaning_map(filepath):

    # Our repo has a map of pre-cleaning
        URL formats (with www. prefixes,
        trailing /'s, etc.)
    # Multiple originals may collapse to
        the same cleaned form so we keep
        the first occurrence.
    map_df = pd.read_csv(filepath)
```

```
    deduped = map_df.drop_duplicates(
        subset='cleaned_url', keep='first'
        )
    url_map = dict(zip(deduped['
        cleaned_url'], deduped['url']))
    print(f"Loaded cleaning map: {len(
        url_map):,} unique cleaned ->
        original entries")
    return url_map

def filter_hijacked_domains(df: pd.
    DataFrame,
     url_map: dict,
     url_col: str = "item",
     match_col: str = "match_1") -> pd.
         DataFrame:


    # We use the URL cleaning map to
        restore the pre-cleaning URL form
        from before parsing.
    def _extract_domain(url: str) -> str:

        restored = url_map.get(url, url)
        if not re.match(r'^https?://',
            restored):
             restored = 'http://' +
                 restored
        domain = urlparse(restored).
            netloc.lower()
        # Strip www. so that www.example.
            com and example.com compare
            equal
        if domain.startswith('www.'):
             domain = domain[4:]
        return domain

    # With that logic, we extract the
        hostname from both the phishing
        URL and its closest legitimate
        match.
    # If they share the same domain the
        phishing URL is likely a hosted
        page on a legitimate site, not an
        impersonation attempt, so we drop
        it.
    phish_domains = df[url_col].apply(
        _extract_domain)
    match_domains = df[match_col].apply(
        _extract_domain)

    same_domain = phish_domains ==
        match_domains
    n_removed = same_domain.sum()

    print(f"Rows where phishing domain ==
         match_1 domain: {n_removed:,}")
    print(f"Rows remaining after filter:
        {len(df) - n_removed:,}")

    return df[~same_domain].reset_index(
        drop=True)

def restore_url(cleaned: str, url_map:
    dict) -> str:

    restored = url_map.get(cleaned,
        cleaned)
    if not re.match(r'^https?://',
        restored):
         restored = 'http://' + restored
    return restored

def extract_features(df: pd.DataFrame,
    url_map: dict,
             url_col: str = "item") -> pd
                 .DataFrame:

    urls = df[url_col].apply(lambda u:
        restore_url(u, url_map))
    hosts = urls.apply(lambda u: urlparse
        (u).netloc.lower())
    paths = urls.apply(lambda u: urlparse
        (u).path)

    feats = pd.DataFrame(index=df.index)
    feats['full_length']      = df[url_col
        ].str.len()
    feats['host_length']      = hosts.str.
        len()
    feats['path_depth']       = paths.
        apply(lambda p: p.strip('/').count
        ('/') + 1 if p.strip('/') else 0)
    feats['dot_count']        = df[url_col
        ].str.count(r'\.')
    feats['hyphen_count']     = df[url_col
        ].str.count('-')
    feats['digit_count']      = df[url_col
        ].str.count(r'\d')
    feats['digit_ratio']      = feats['
        digit_count'] / feats['full_length
        ']
    feats['subdomain_depth'] = hosts.str.
        count(r'\.')
    return feats

def create_labels(df: pd.DataFrame,
    quantile: float = 0.75) -> tuple: # 75
     percentile being our chose default

    threshold = df['score_1'].quantile(
        quantile)
```

```
    df = df.copy()
    df['is_impersonating'] = (df['score_1
        '] >= threshold).astype(int)
    label_counts = df['is_impersonating'
        ].value_counts()

    print(f"Impersonation threshold ({
        quantile:.0%} percentile): {
        threshold:.4f}")
    print(f"\nClass distribution:")
    print(f"  0 (not impersonating): {
        label_counts.get(0, 0):,}")
    print(f"  1 (impersonating):     {
        label_counts.get(1, 0):,}")
    print(f"  Ratio: {label_counts.get(1,
         0) / len(df):.2%}")

    return df, threshold, label_counts

def plot_score_distribution(df_sim,
    threshold, label_counts):

    fig, axes = plt.subplots(1, 2,
        figsize=(13, 5))

    axes[0].hist(df_sim['score_1'], bins
        =60, color='midnightblue',
        edgecolor='white', alpha=0.85)
    axes[0].axvline(threshold, color='red
        ', ls='--', lw=1.5,
             label=f'Threshold = {
                 threshold:.3f}')
    axes[0].set_title('Figure 1:
        Distribution of score_1 (Best-
        Match Cosine Similarity)')
    axes[0].set_xlabel('Cosine Similarity
        ')
    axes[0].set_ylabel('Frequency')
    axes[0].legend()

    # The below colors are chosen to
        align a bit with GS Branding
    colors = ['midnightblue', '
        darkgoldenrod']
    axes[1].bar(['Not Impersonating (0)',
         'Impersonating (1)'],
        [label_counts.get(0, 0),
            label_counts.get(1, 0)],
        color=colors, edgecolor='white')
    axes[1].set_title('Figure 2: Binary
        Label Distribution')
    axes[1].set_ylabel('Count')

    plt.tight_layout()
    plt.show()

def plot_feature_distributions(features,
    df_sim):

    feature_names = features.columns.
        tolist()
    plot_df = features.copy()
    plot_df['is_impersonating'] = df_sim[
        'is_impersonating']

    fig, axes = plt.subplots(2, 4,
        figsize=(18, 9))
    axes = axes.flatten()
    for i, feat in enumerate(
        feature_names):
        sns.boxplot(data=plot_df, x='
            is_impersonating', y=feat, ax=
            axes[i],
            hue='is_impersonating',
                palette=['darkblue', '
                darkgoldenrod'],
            legend=False, showfliers=
                False)
        axes[i].set_title(feat)
        axes[i].set_xlabel('')

    fig.suptitle('Figure 3: Feature
        Distributions by Impersonation
        Label', fontsize=14, y=1.01)
    plt.tight_layout()
    plt.show()

def split_data(features, df_sim,
    test_size, random_state):

    X = features.values
    y = df_sim['is_impersonating'].values

    X_train, X_test, y_train, y_test =
        train_test_split(
        X, y, test_size=test_size,
            random_state=random_state,
            stratify=y
    )

    print(f"Training set: {X_train.shape
        [0]:,} samples")
    print(f"Test set:     {X_test.shape
        [0]:,} samples")
    print(f"Train class 1 ratio: {y_train
        .mean():.2%}")
    print(f"Test  class 1 ratio: {y_test.
        mean():.2%}")
    return X_train, X_test, y_train,
        y_test

def train_baseline_svm(X_train, y_train,
```

```
    X_test, y_test, random_state):

    svm_pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='rbf',
            class_weight='balanced',
            probability=True,
             random_state=random_state))
    ])
    svm_pipeline.fit(X_train, y_train)
    y_pred = svm_pipeline.predict(X_test)

    print("Baseline SVM (RBF) - Test Set
        Results")
    print(classification_report(y_test,
        y_pred,
          target_names=['Not Impersonating
              ', 'Impersonating']))
    return svm_pipeline, y_pred

def tune_svm(svm_pipeline, X_train,
    y_train, X_test, y_test, random_state,
      n_iter=12):

    from scipy.stats import loguniform

    param_distributions = {
        'svm__C':      loguniform(1e-1, 1
            e3),
        'svm__gamma': loguniform(1e-3, 1
            e0),
    }
    cv = StratifiedKFold(n_splits=5,
        shuffle=True, random_state=
        random_state)

    search = RandomizedSearchCV(
        svm_pipeline, param_distributions
            ,
        n_iter=n_iter, cv=cv, scoring='f1
            ',
        n_jobs=-1, verbose=1,
        random_state=random_state,
        return_train_score=True
    )
    search.fit(X_train, y_train)

    print(f"\nBest parameters: {search.
        best_params_}")
    print(f"Best CV F1 score: {search.
        best_score_:.4f}")

    best_svm = search.best_estimator_
    y_pred_tuned = best_svm.predict(
        X_test)

    print("\nTuned SVM (RBF) - Test Set
        Results")
    print(
        classification_report(
            y_test,
            y_pred_tuned,
            target_names=['Not
                Impersonating', '
                Impersonating']
        )
    )
    return best_svm, y_pred_tuned, search

def plot_confusion_matrix(y_test,
    y_pred_tuned):

    fig, ax = plt.subplots(figsize=(6, 5)
        )
    ConfusionMatrixDisplay.
        from_predictions(
        y_test, y_pred_tuned,
        display_labels=['Not
            Impersonating', 'Impersonating
            '],
        cmap='Blues', ax=ax
    )
    ax.set_title('Figure 4: Confusion
        Matrix - Tuned RBF SVM')
    plt.tight_layout()
    plt.show()

def plot_roc_curve(best_svm, X_test,
    y_test):

    y_prob = best_svm.predict_proba(
        X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test,
        y_prob)
    roc_auc = auc(fpr, tpr)

    fig, ax = plt.subplots(figsize=(7, 6)
        )
    ax.plot(fpr, tpr, lw=2, label=f'SVM (
        AUC = {roc_auc:.3f})', color='
        midnightblue')
    ax.plot([0, 1], [0, 1], 'k--', lw=1,
        alpha=0.4, label='Random Chance')
    ax.set_xlabel('False Positive Rate')
    ax.set_ylabel('True Positive Rate')
    ax.set_title('Figure 5: ROC Curve -
        Tuned SVM')
    ax.legend(loc='lower right')
    ax.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
    print(f"AUC: {roc_auc:.4f}")
```

```
    return roc_auc

def analyze_feature_importance(X_train,
    y_train, X_test, y_test,
        feature_names, random_state):

    linear_pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='linear',
            class_weight='balanced',
            probability=True,
             random_state=random_state))
    ])
    linear_pipeline.fit(X_train, y_train)

    y_pred_linear = linear_pipeline.
        predict(X_test)
    print("Linear SVM - Test Set Results"
        )
    print(classification_report(y_test,
        y_pred_linear,
          target_names=['Not Impersonating
              ', 'Impersonating']))

    coefs = linear_pipeline.named_steps['
        svm'].coef_[0]
    coef_df = pd.DataFrame({'feature':
        feature_names, 'coefficient':
        coefs})
    coef_df = coef_df.sort_values('
        coefficient')

    fig, ax = plt.subplots(figsize=(9, 5)
        )
    colors = ['darkgoldenrod' if c > 0
        else 'midnightblue' for c in
        coef_df['coefficient']]
    ax.barh(coef_df['feature'], coef_df['
        coefficient'], color=colors,
        edgecolor='white')
    ax.axvline(0, color='black', lw=0.8)
    ax.set_xlabel('SVM Coefficient (
        standardised)')
    ax.set_title('Figure 6: Linear SVM
        Feature Coefficients\n(positive ->
         impersonating, negative -> not
        impersonating)')
    plt.tight_layout()
    plt.show()
    return coef_df

def main():


    # Data Loading
    print("DATA LOAD:")
    print("-" * 70)
    df_sim = load_data(TOP5_PHISH_FILE)
    url_map = load_cleaning_map(
        URL_MAP_FILE)
    print("\n\n\n")

    # Domain Hijacking Filter
    print("DOMAIN HIJACKING FILTER:")
    print("-" * 70)
    df_sim = filter_hijacked_domains(
        df_sim, url_map)
    print("\n\n\n")

    # Preprocessing
    print("PREPROCESSING:")
    print("-" * 70)
    features = extract_features(df_sim,
        url_map)
    feature_names = features.columns.
        tolist()
    print(f"Extracted {features.shape[1]}
         features for {features.shape
        [0]:,} URLs\n")
    display(features.describe().round(3))
    print("\n\n\n")
    df_sim, threshold, label_counts =
        create_labels(df_sim)
    print("\n\n\n")

    # Exploratory Data Analysis
    print("EXPLORATORY DATA ANALYSIS:")
    print("-" * 70)
    plot_score_distribution(df_sim,
        threshold, label_counts)
    plot_feature_distributions(features,
        df_sim)
    print("\n\n\n")

    # Modeling
    print("MODELING:")
    print("-" * 70)
    X_train, X_test, y_train, y_test =
        split_data(
         features, df_sim, TEST_SIZE,
            RANDOM_STATE
    )
    print("\n\n\n")

    svm_pipeline, y_pred =
        train_baseline_svm(
         X_train, y_train, X_test, y_test,
             RANDOM_STATE
    )

    best_svm, y_pred_tuned, grid_search =
         tune_svm(
```

```
        svm_pipeline, X_train, y_train,
            X_test, y_test, RANDOM_STATE
    )
    print("\n\n\n")

    # Evaluation
    print("EVALUATION:")
    print("-" * 70)
    plot_confusion_matrix(y_test,
        y_pred_tuned)
    plot_roc_curve(best_svm, X_test,
        y_test)
    analyze_feature_importance(
        X_train, y_train, X_test, y_test,
            feature_names, RANDOM_STATE
    )
    print("\n\n\n")

    print("-" * 70)
    print("PIPELINE COMPLETE")
    print("-" * 70)

# Execute the full pipeline --
main()
```

D. Code of Classification Model (Converted from .ipynb)

```
import os

if not os.path.exists('
    DS_MLProject_ColabIntegration'):
  !git clone https://github.com/ns15468-
      gasou/DS_ML_Project_ColabIntegration
      .git
os.chdir('DS_ML_Project_ColabIntegration/
    Project/notebooks')
print("Working directory", os.getcwd())

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import
    StandardScaler
from sklearn.cluster import KMeans
import numpy as np
from urllib.parse import urlparse

phish = pd.read_csv('../data/
    processed_input/phishtank_cleaned.csv'
    )

other = phish[phish['target']=='Other'].
    copy()
print(f"total Phishtank rows: {len(phish)
    }")
print(f"'Other' rows: {len(other)}")
other.head()

phish['host'] = phish['url'].apply(lambda
     x: urlparse(x).netloc.lower())
phish['full_length'] = phish['url'].str.
    len()
phish['host_length'] = phish['host'].str.
    len()
phish['path'] = phish['url'].apply(lambda
     x: urlparse(x).path)
phish['path_depth'] = phish['path'].apply
    (lambda p:
p.strip('/').count('/') + 1 if p.strip('/
    ') else 0)
phish['dot_count'] = phish['url'].str.
    count(r'\.')
phish['hyphen_count'] = phish['url'].str.
    count('-')
phish['digit_count'] = phish['url'].str.
    count(r'\d')
phish['digit_ratio'] = phish['digit_count
    '] / phish['full_length']
phish['subdomain_depth'] = phish['host'].
    str.count(r'\.')

other = phish[phish['target'] == 'Other'
    ].copy()
print(other.shape)

features = ['host_length', 'path_depth',
    'dot_count', 'hyphen_count', '
    digit_ratio', 'subdomain_depth']

X = other[features].dropna()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"Feature matrix shape: {X_scaled.
    shape}")

inertia = []
k_range = range(2,11)

for k in k_range:
  km = KMeans(n_clusters=k, random_state
      =777, n_init=10)
  km.fit(X_scaled)
  inertia.append(km.inertia_)

plt.figure(figsize=(8,4))
plt.plot(k_range, inertia, marker='o')
plt.title('Elbow Method- Optimal K')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.tight_layout
plt.show()
```

```
km = KMeans(n_clusters=4, random_state
    =777, n_init=10)
other['cluster'] = km.fit_predict(
    X_scaled)

print(other['cluster'].value_counts())

cluster_summary = other.groupby('cluster'
    )[features].mean().round(2)
print(cluster_summary)

for c in range(4):
  print(f"\n--- Cluster {c} ---")
  print(other[other['cluster']== c]['url'
      ].sample(5, random_state=777).values
      )

samples = []
for c in range(4):
      label = cluster_names[c]
      urls = other[other['cluster'] == c
          ]['url'].sample(3,
  random_state=42).values
      for url in urls:
          samples.append({'Cluster':
              label, 'Example URL': url
              [:60] +
  '...' if len(url) > 60 else url})

sample_df = pd.DataFrame(samples)

fig, ax = plt.subplots(figsize=(12, 5))
ax.axis('off')
table = ax.table(cellText=sample_df.
    values, colLabels=sample_df.columns,

           loc='center', cellLoc='left')
table.auto_set_font_size(False)
table.set_fontsize(8)
table.auto_set_column_width(col=list(
    range(len(sample_df.columns))))
plt.title('Sample URLs by Cluster', pad
    =20)
fig.patch.set_visible(False)
plt.savefig('cluster_samples.png', dpi
    =150, bbox_inches='tight',
transparent=True)
plt.show()

cluster_names = {0: 'Platform Abuse (
    Simple)',
          1: 'Platform Abuse (Complex)',
          2: 'Token/Long URL Abuse', 3: '
              Shortener/Redirector'}

other['cluster_label'] = other['cluster'
    ].map(cluster_names)
other['cluster_label'].value_counts().
    plot(kind='barh', figsize=(8,4),
              color='steelblue')
plt.title('PhishTank "Other" - Cluster
    Distribution')
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```